Page No.
Date : / /

Lecture-7

Function & Array

```
return_type function_name (parameter1, parameter2, ...){
    // code

    return value;
}
```

→ Why We need function :

Let us understant by example

You have given 8 number to check that it is even or odd.

You can Write the same code for all the input.

Here can Write same code many time.

To minimize this problem we use function.

A function We can Write once for a specific work and use the same code for all the input value.

Now, We understand how to Write function.

Function for checking a number is Even or odd.

Teacher's Signature.....

```
#include <iostream>
using namespace std;

Void isEvenOdd (int num){

    if((num % 2) == 0)
        Cout << " Even" << endl;

    else
        Cout <<"odd" <<endl;

  return;
}

int main () {
    int num1, num2, num3, num4;
    Cin >> num1 >> num2 >> num3 >> num4;
        isEvenOdd (num1);
        isEvenOdd (num2);
        isEvenOdd (num3);
        isEvenOdd (num4);

  return 0;
}
```

In this code, we can check for four number that it is Even or Odd.

With the help of function, we can write once and use it for many time.

Teacher's Signature.....

Page No.
Date / /

* Take an another Example : function for print table :

We have to Write table for the given number.

```
Void printTable (int num) {

    for (int i = 1; i <= 10; i++) {
        Cout << num << "*" << i << " = " << num * i;
    }

  return ;
}

int main() {
    int n;
    cin >> n;
    table(n);
    table (13);
  return 0;
}
```

* factorial of a number

$5! = 5 \times 4! \rightarrow 24$ (120)

$4 \times 3! \rightarrow 6$

$3 \times 2! \rightarrow 2$

$2 \times 1! \rightarrow 1$

Teacher's Signature.....

```
int factorial (int num){
        int ans = 1;
    for (int i=1; i<= num; i++ ){
            ans = ans * i;
        }
    return ans;
}

int main() {
    int num;
    cin >> num1;

  int fact = factorial (num1);
   cout << fact;

return 0;
}
```

* function overloading :

In our previous problem we can give a fix number of parameter.

In we write a function to add 2 number then we give 2 parameter.
And in same function, we wont to add 3 number, then it can't be possible.

Teacher's Signature.....

So, for this problem, We can use Method overloading.

The name of the function is same and the parameter is difference.

We have to Write Code for add, Where we wanto add 2 number, in some case we add 3 number and in few we add upto four number.

```
int add (int num1, int num2) {
        int ans = num1 + num2;
    return ans;
  }
float
~~int~~ add ( float num1, float num2) {
        float ans = num1 + num2;
  return ans;
  }
int add ( int num1, int num2, int num3) {
        int ans = num1 + num2 + num3;
    return ans;
  }

int main() {
    int a = add (9, 6);
      Cout << a;
        Cout << endl;
    int b = add ( 24, 9, 13);
      Cout << b;
      Cout << endl;
```

Teacher's Signature.....

```
    float c = 9.6 + 11.3;
      Cout << c;
      Cout << endl;
  return 0;
}
```

The above concept of function are known as function called by value.

Arrays

* A person want to store Apple, he rent a storeroom in nearby building. Room No: 109.

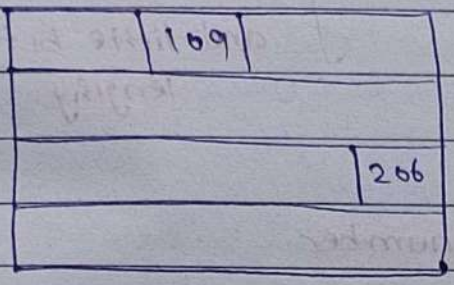
The person have coming more Apple, he rent one more storeroom is same building. Room No: 206.

The sale of apple is increased rapidly. He then rent 20 room in same building. But he want the room which are all continous.

The owner has alloted room no. 101 to 120 to the person.

Teacher's Signature.....

Previously:



In this, person has to remember the room number.

Now:

101	102	103	104	105	106	107	108
109	110	111	112	113	114	115	116
117	118	119	120				206

It this, person has to remember the room number of 1st room only. and rest of the room are continous.

Array: Collection of same datatype.

Why We need?

Example: Sum of 3 number

```
int a = 3
int b = 10
int c = 13
```

Easily We do

Sum of 10 number

```
int a, b, c, d, e, f ---
```

easily We do

Page No.:
Date : / /

Sum of 100 number
Make 100 variable
then sum
} takes some time and little bit lengthy.

Sum of 1000 number
Make 1000 variable
It take more time and very lengthy task.

So, for storing the same datatype element we use array.

For storing 1000 number:
int a1, a2, a3, a4, a5, ----
By this way we can store.

By seeing deeply,
only a①, a②, a③, a④ ---
this changes.

So, we can done this in another way by array.

a[i] -> a[0], a[1], a[2] ---

Array is a Collection of same data type.
It stored at Configous location.

Teacher's Signature.....

Page No.:
Date: / /

Type of element is same in array.

arr[] = type → int
→ float
→ char.

Declaration :

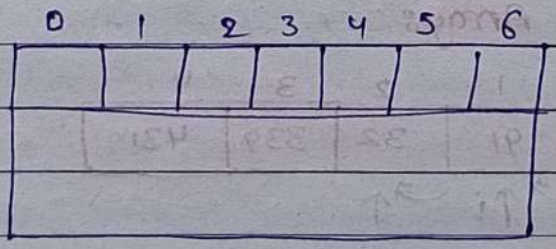
int arr[7]; → It takes 7 space in Memory to store integer.

0	1	2	3	4	5	6
9	2	6	0	-3	18	26

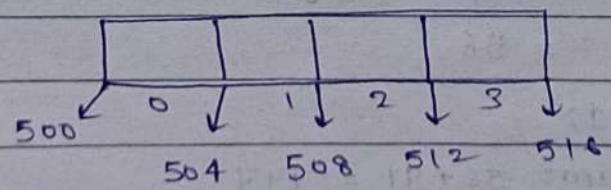
→ Why indexing is start from 0?

arr[0] = 9;
cout << arr[0]; print 9.

In Computer, it uses byte addressable.



Integer type array



Memory address

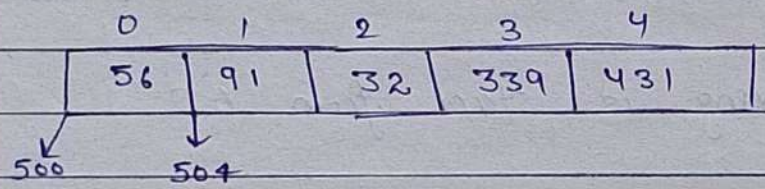
Teacher's Signature.....

We use indexing from 0 because for accessing the address in 1 based indexing can take more time.

In 1 based indexing:
Initial address + (index - 1) * size;

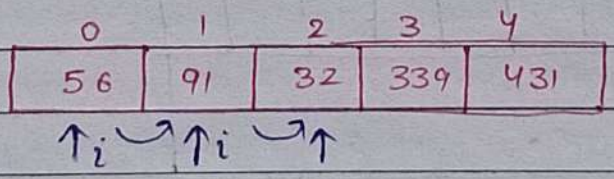
In 0 based indexing:
Initial address + index * size;

int arr[5];



Integer can take 4 byte to store an integer.

* Sum of an Array:



Size = 5

sum = 0

sum = sum + a[i]

= 56

i = 1;

sum = 56 + 91 = 147

i++;

i = 2

Teacher's Signature.....

Page No.:
Date: / /

sum = 147 + 32 = 179

i++ ; i = 3

sum = 179 + 339 = 518

i++ ; i = 4

sum = 518 + 431 = 949

i++ ; (i = 5)

↳ Not present

loop break;

print sum = 949.

Code:

```
int main () {
    int arr[5];
    arr[0] = 56 , arr[1] = 91, arr[2] = 32;
    arr[3] = 339, arr[4] = 431;
    int sum = 0;
for (int i = 0; i<5; i++) {
    sum = sum + arr[i];
}

cout << sum;

return 0;
}
```

Teacher's Signature:.....

Page No.:
Date: / /

Lecture - 8

(problems on Array)

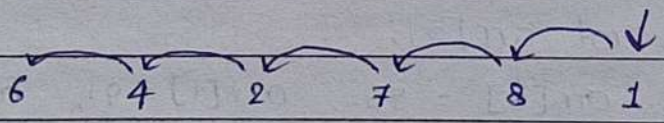
problem 1: Reverse of an Array

Index :	0	1	2	3	4	5
	6	4	2	7	8	1

Approach 1

Traverse the array and print array from the last.

Array after reverse:



Arr_rev = {1, 8, 7, 2, 4, 6}

```
for (int i= arr.length -1 ;  i>=0 ; i--)
        Cout <<arr[i] <<" " ;
```

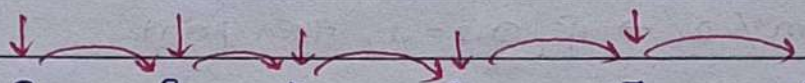
Problem 2: Maximum Element of an array

0	1	2	3	4	5
2	3	11	8	7	-2

Array includes with negative number also.

So, initialise MAX as INT_MIN:

Now, traverse the array and update Max is number bigger than Max found.



2 > INT_MIN → MAX = 2

3 > 2 → Max = 3

11 > 3 → Max = 11

8 < 11 → No change

7 < 11 → No change

-2 < 11 (No change)

Max = ~~INT_MIN~~ ~~2~~ ~~3~~ 11

Code:

```
int arr = { 2, 3, 11, 8, 7, -2};
    int largest = INT_MIN;
for (int i = 0; i < arr.size(); i++) {
        if (arr[i] > largest)
            largest = arr[i];

Cout << largest;
```

Teacher's Signature.....

problem 3: print all odd number of array:

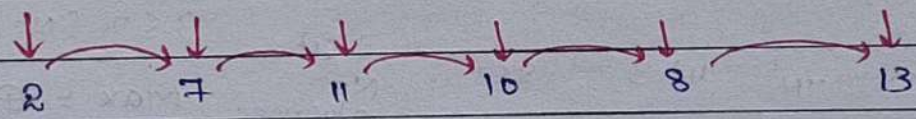
arr: {2, 7, 11, 10, 8, 13}

for odd:

condition

↳ When we divide by 2 remaineder comes 1.

① traverse the array and check all element by num%.2 → if ans = 1, then print otherwise even → do not print.



2%.2 = 0	7%.2 = 1	11%.2 = 1	10%.2 = 0	8%.2 = 0	13%.2 = 1
Even	odd	odd	Even	Even	odd
Not print	print	print	Not print	Not print	print

ans = 7 11 13

code:

```
int arr = {2, 7, 11, 10, 8, 13};
  for(int i = 0; i < arr.size(); i++){
      if(arr[i]%.2 == 1 || arr[i]%.2 == -1){
          Cout << arr[i] << " ";
      }
  }
```

Teacher's Signature.....

problem 4 : print all prime number from array.

arr = { 2, 3, 7, 11, 13 }

arr = { 2, 3, 4, 8, 7, 11, 16, 19 }

for prime :

↳ for all number, we have to check that it is divisible by 1 & self.

↳ the number have more than 2 factor the it is not prime.

8 : ~~1 x 2 x~~ 1, 2, 4, 8

More than 2 → not prime.

13 : 1, 13

Exactly 2 → Prime.

Code :

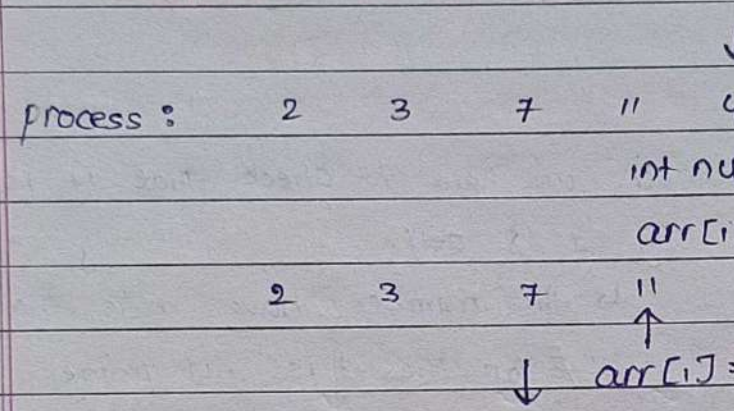
```
int arr[] = {2, 3, 4, 8, 7, 11, 16, 19};
    for (int i = 0; i<arr.size(); i++) {
            prime(arr[i]);
    }
Void prime (int num) {
        if (num < 2)
            return;
    for(int i = 2; i<=num-1; i++){
            if (num % i == 0)
                return;
        }
    Cout<<num<<" ";
return;
}
```

Teacher's Signature.....

Problem 5: Rotate 1 place of Array:

arr: { 2 3 7 11 4 }

Output: 4 2 3 7 11



Ans: 4 2 3 7 11

Code:

```
int arr[] = { 2, 3, 7, 1, -11, 8, 13 };
        int num = arr[arr.size()-1];

for (int i = arr.length - 2 ; i >= 0; i--) {
        arr[i+1] = arr[i];
    }
    arr[0] = num;
for (int i = 0;  i < arr.length; i++) {
        cout << arr[i] << endl;
    }
```

Teacher's Signature.....

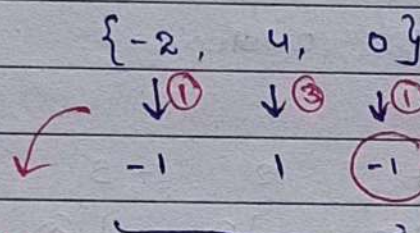
Page No.
Date / /

problem 6 : Maximum step to make the product 1

Example : {-2, 4, 0}

For product = 1

Even no. of (-1) and Even/odd number of 1.



→ इसको -1 इसलिए बनाए है क्योंकि मेरे पास Negative number एक ही था, ↓ odd.

$-1 \times 1 \times -1 = 1$

→ If we have odd number of negative number then, we make one zero as (-1).

↳ If 0 is not present then make a negative number to 1.

↳ If 0 is present add 1 step

↳ If 0 is not present add 2 step

-ve → -1

+ve → 1

0 → -ve / +ve (-1/+1)

↓

If 0 is not present then,

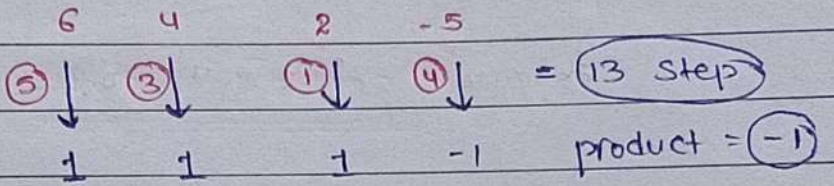
in few case (-ve → 1)

Even No. of (-1) x odd/even number of 1 = 1.

Teacher's Signature.....

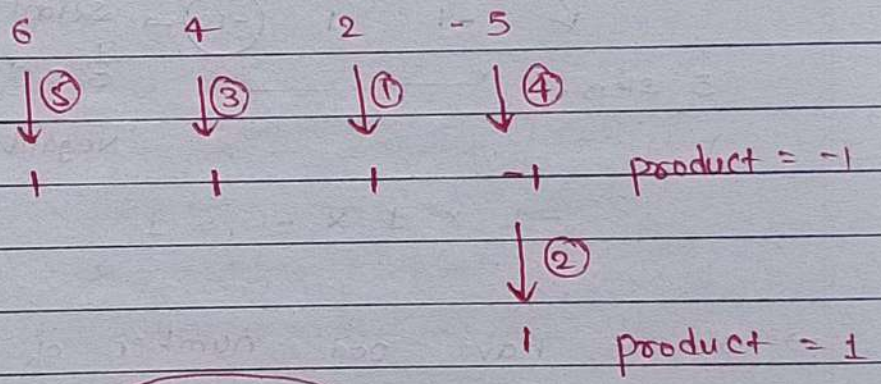
Page No.:
Date: / /

Ex:



So, We make 1 (-1) as 1.

then add 2 step



Step = 15

code:

```
int count_zero = 0;
int step = 0;
int multiplier = 1;

for (int i = 0; i < arr.length; i++){
    if (arr[i] > 0) {
        step = step + (arr[i] - 1);
        Multiplier = Multiplier * 1;
    }
    else if ( arr[i] < 0){
        step = step + (-1 - arr[i]);
        Multiplier = Multiplier * (-1);
    }
```

Teacher's Signature.....

```
        else {
                step = step + 1;
                count_zero++;
            }

        if (multiplier == 1 || count_zero > 0) {
                ans = step;
            }
        else {
                ans = step + 2;
            }
        return ans;
    }
```

if Multiplier = 1
↳ then NO problem
↳ return step.

if Multiplier != 1
↳ Check count_zero > 0
↳ if count_zero > 0
↳ return step.
↳ if count_zero = 0
↳ then step = step + 2;